CogniGuide Phase 2: Progress Report

Stage 1 (Flask /chat API): complete and tested | 16 October 2026

1. What we did

Until now the tutor was only a Python file (**socratic_agent.py**) that could be run by hand. An app cannot call a Python file, so we built a small web server in front of it using **Flask**. The app sends a maths problem and the conversation so far; the server asks the tutor for its next message and sends it back. The tutor itself was **not changed**, so the Phase 1 baseline evaluation still uses the original.

```
App (student's message)
   |  JSON over HTTP
   v
Flask server  (api.py)  - checks the request
   |
   v
socratic_response()  (Phase 1, unchanged)  ->  Gemini
   |
   v
Tutor's guiding question goes back to the app
```

2. What we use

Tool	What it is	Why
Python + Flask	A Python library that turns functions into web addresses	The app cannot call a Python file directly; Flask gives it addresses to call
flask-cors	A small add-on for Flask	Lets a browser-based frontend on a different address call our server (CORS)
Gemini API	The AI that writes the tutor's messages	Already working in Phase 1; model name is set in .env
python-dotenv	Reads secrets from the .env file	Keeps keys out of the code and out of GitHub
Supabase (next)	Online database	Will store anonymous student IDs and chat logs (Stages 2 and 3)

3. The output: what the server offers

Address	What it does	Reply
GET /health	Checks the server is alive	{"status": "ok"}
POST /chat	Takes the problem and the conversation, returns the tutor's next message	{"reply": "...", "session_id": ...}

Bad requests (missing problem, wrong role, text too long, bad JSON) get a clear error and never reach Gemini. Every error has the same shape: `{"error": {"code": ..., "message": ...}}`. The full agreement is in **docs/api_contract.md**.

4. Test results (live, with the real Gemini)

Run on 16 October 2026 with `experiments/phase2_adaptive_agent/test_chat.py`, using the Natalia clips problem:

#	What the student did	Status	What the tutor did	Result
1	First message, no history	200	Opened with a guiding question about how to find how many she sold in May	Pass
2	"I don't know how to start."	200	Reassured the student, pointed to the 48 clips and "half as many", asked which operation finds half	Pass
3	"Just tell me the answer."	200	Did not give the answer; made it easier with a 10-clips example and asked a question	Pass

Also checked earlier with a stand-in for Gemini: /health, bad JSON, missing problem, wrong role, history not a list, text too long, missing API key (500), wrong method (405) and unknown URL (404) all returned

the status and error format in the contract. Still to check in a browser frontend: the CORS header.

5. What we changed in the repo

File	Status	What it is
backend/app/api.py	New	The server: /health and /chat, request checking, error handling
backend/app/db.py	New	Supabase helper (Stage 3 code; not active until Supabase is set up)
backend/sql/schema.sql	New	Blueprint for the four database tables and privacy rules (Stage 2, not yet run)
docs/api_contract.md	New	Exact request/reply/error format for the frontend
experiments/phase2_adaptive_agent/test_chat.py	New	Script that sends three test messages to the server
requirements.txt	Changed	Added flask, flask-cors, supabase
backend/app/models/socratic_agent.py	**Not changed**	Phase 1 baseline stays exactly as it was

6. Decisions we made

- **api.py only calls socratic_response().** No change to the Phase 1 agent, so the baseline evaluation stays valid.
- **The tutor is stateless.** The app sends the whole conversation every time; saving to the database is separate.
- **One error format** for every error, with codes invalid_request (400), not_found (404), method_not_allowed (405), server_error (500).
- **Limits:** each text up to 2000 characters, history up to 50 messages (easy to change in api.py).
- **Anonymous student codes only** (letters, digits, _ and -). Names and emails are rejected, because our participants are 13 to 18.
- **Saving must never break the chat.** If the database is down, the student still gets the tutor's reply.
- **CORS is open to everyone for now** while we build. It must be restricted to the real frontend before the user study (Phase 3).
- **GEMINI_MODEL lives only in .env** (set by the latest commit). The server needs both GEMINI_API_KEY and GEMINI_MODEL.

7. Where we are

Stage	Status
1. Flask /chat API	**Done and tested live**
2. Supabase database	Next. Project not created yet; schema.sql is ready to run
3. Saving every chat	Code written (db.py, api.py); can only be tested after Stage 2
4 to 9	Not started (adaptive reasoning, scaffolding, reflection prompts, dashboard, testing, report)

8. What the team should check or decide

- Read **docs/api_contract.md**. It is the document the frontend will be built against.
- **Frontend: Flutter or React?** The proposal and the repo folders (frontend/lib/screens, widgets) point to Flutter, but the project description says React. The server works with either, but this decides what the Figma designs get built in. A change from the proposal needs supervisor sign-off.
- Confirm the server starts on everyone's computer: create a virtual environment, `pip install -r requirements.txt`, then run `python -m app.api` from the backend folder.

9. How to run it

```
python -m venv venv                  (one time only)
venv\Scripts\activate                (Windows)   |   source venv/bin/activate   (Mac)
pip install -r requirements.txt
cd backend
python -m app.api                    (leave running)

Open  http://127.0.0.1:5000/health   ->  {"status": "ok"}
In a second terminal, from the repo root:
python experiments/phase2_adaptive_agent/test_chat.py
```

The .env file in the repo root needs GEMINI_API_KEY and GEMINI_MODEL. It is never uploaded to GitHub.

10. What comes next

Stage 2: create the Supabase project and run schema.sql (four tables: students, sessions, messages, indicators, with row-level security on). Stage 3: switch on saving, so every chat is stored under an anonymous student code and a session id.